

SPICE: Automated Scientific Program Repair through Contract Probing and Self-Evolving

Xiubo Zhang

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

Feifei Niu

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

Abstract—Automated program repair (APR) for scientific software is uniquely challenging because candidate patches that pass conventional unit tests often violate underlying scientific invariants and physical constraints. We propose SPICE, a multi-agent framework tailored for scientific software repair that integrates contract-aware probing, dual-expert patch synthesis, and domain-constrained evaluation. To overcome the prevalent false-green-light issue, SPICE employs a Prober to extract explicit scientific contracts and generate patch-independent executable probes ($T_P \perp \mathcal{P}$). A Patcher then decouples domain reasoning from structural code editing via a dual-expert protocol leveraging sibling call-site mappings, after which a Reviewer evaluates candidate patches across six quality dimensions while strictly enforcing hard physical constraints. Extensive evaluation on scientific software repair benchmarks demonstrates that SPICE achieves an average private test pass rate of 57.1%, significantly outperforming state-of-the-art baselines including Agentless (43.8%, a 13.3% absolute gain), MAGIS (38.2%), and DeepSeek Pro (35.2%). Beyond individual repair tasks, SPICE consolidates verified experience into reusable memory for continuous self-evolution, and is deployed as a VS Code extension to deliver plug-in services that help non-computer-science scientists rapidly diagnose and repair domain-specific code.

Index Terms—Automated Program Repair, Scientific Software, Large Language Models, Multi-Agent Systems, Self-Evolution

I. INTRODUCTION

Automated Program Repair (APR) has evolved rapidly from deep learning paradigms to Large Language Model (LLM)-based architectures. However, applying LLM-based APR to scientific software exposes critical failure modes, such as severe overfitting and contract violations [1]. As domain codebases expand and increasingly rely on LLM-generated routines, undetected silent errors undermine scientific validity [2]. Empirical findings on SWE-bench-Science show that while scientific context provides essential semantic constraints beyond local syntax, directly injecting domain knowledge into monolithic coding agents yields inconsistent trade-offs—either inducing severe token overheads or causing *cognitive anchoring* and performance degradation [2].

To address this challenge, we propose a multi-agent APR framework based on a novel *Prober-Patcher-Reviewer*

paradigm. By synthesizing structured specifications and executing deep repository exploration, our approach efficiently extracts latent scientific invariants as actionable evidence, guiding LLMs toward scientifically sound and contract-compliant program repairs.

The contributions of this paper are:

Contributions

- We propose an automated program repair algorithm based on multi-agent writing, and experiments demonstrate that our method improves program repair performance.
- Building on the previous point, we equip our agents with self-evolving algorithms and storage structures—capabilities that align closely with the need for scientific coding professionals to deploy agent assistants locally.
- We are packaging the method as a VS Code extension and providing a corresponding harness, making it easier for scientists to use this algorithm. (Not yet fully developed / Incomplete)

II. MOTIVATING EXAMPLES

In SWE-bench-Science [2], scientific context is shown to affect repair performance in model-dependent ways. On the 91-task separable subset, DeepSeek-V4-flash achieves higher Pass@1 when provided with scientific context, albeit at the cost of substantially increased token consumption. Conversely, GPT-5.6-sol consumes fewer tokens but exhibits a degraded Pass@1 rate. Task-level analysis reveals that while scientific context offers crucial semantic constraints beyond localized code symptoms, it also introduces risks such as cognitive anchoring and over-reliance on supplied explanations. These findings raise a fundamental research question: *Rather than directly injecting raw scientific knowledge into a monolithic agent, can a lightweight, domain-aware Prober distill relevant scientific contracts and invariants into actionable evidence for a stronger Patcher?*

A representative failure case from `pvlb` illustrates the necessity of such targeted evidence:

This research was developed with the assistance of Large Language Models (LLMs) and AI-assisted tools for code debugging and text refinement.

SPICE: Agentic Automated Program Repair for Scientific Computing



Fig. 1. Overview of the Architecture of Our Multi-agent System

Example 1. Two timezone-aware time series denote identical physical instants but carry distinct local labels (e.g., UTC vs. America/Los_Angeles). Although execution yields a `workflow_completed` status (exit code 0), clear-sky Direct Normal Irradiance (DNI) differs by 0.120 W/m^2 because the day-of-year calculation relies on local calendar dates. Consequently, this bug eludes standard execution checks, requiring an explicit invariant probe to enforce consistency across physically equivalent temporal representations.

Research Questions

- **RQ1** Can we enable powerful models to also benefit from domain and scientific knowledge?
- **RQ2** To what extent do boundary conditions, contracts, and invariants influence program repair, and how do we identify them?
- **RQ3** Will multi-agent systems necessarily outperform single-agent systems in code repair tasks?
- **RQ4** Can we enable automated code repair tools to self-evolve when solving scientific problems?

III. METHODOLOGY

SPICE decouples invariant discovery from patch synthesis via three specialized agents: (1) **Prober**: Discovers contracts and synthesizes tests; (2) **Patcher**: Synthesizes candidate patches via dual-expert generation; and (3) **Reviewer**: Evaluates repairs under physical constraints. Formally, given repository artifacts $\mathcal{A}$, baseline tests T_0 , and domain knowledge $\mathcal{K}$, SPICE generates an optimal patch P^* :

$$SPICE(\mathcal{A}, T_0, \mathcal{K}) \longrightarrow P^*. \quad (1)$$

A. Probing: Contract Discovery and Test Generation

To ensure test independence ($T_P \perp \mathcal{P}$), Probing executes prior to patch synthesis:

$$\text{Probe}(\mathcal{A}, T_0, \mathcal{K}) \longrightarrow (\mathcal{C}, \mathcal{P}_{\text{spec}}, T_P), \quad (2)$$

where $\mathcal{C}$ denotes contracts, $\mathcal{P}_{\text{spec}}$ probe specs, and T_P synthesized test suite. A scientific contract $c \in \mathcal{C}$ is formalized as $c = (r, s, o, \omega, \mathcal{E}_c)$, encoding required behavior r , scope s , observable state o , oracle tolerance ω , and codebase evidence $\mathcal{E}_c$.

Probes $p = (c, \tau, \iota, \omega, \pi) \in \mathcal{P}_{\text{spec}}$ instantiate tests ($p \rightarrow \{t_1, \dots, t_m\}$) across five strategies: *Direct*, *Boundary*, *Invariant*, *Metamorphic*, and *Interaction*. Augmented tests $T^+ = T_0 \cup T_P$ measure Scientific Contract Coverage (SCov):

$$SCov(T^+, \mathcal{C}) = \frac{|\{c_i \in \mathcal{C} : \text{Exercised}(T^+, c_i)\}|}{|\mathcal{C}|}. \quad (3)$$

Test utility $U(t)$ balances coverage gain, hypothesis differentiation $\text{Disc}(t)$, and execution cost $\text{Cost}(t)$:

$$U(t) = \frac{\Delta SCov(t) + \lambda \cdot \text{Disc}(t)}{\text{Cost}(t)}. \quad (4)$$

B. Dual-Expert Architecture for Patch Synthesis

The Patcher decouples domain reasoning from code generation: (1) **Expert B (Domain Specialist)**: Extracts grounded evidence $\mathcal{E}_{\text{dom}}$ and builds a *Sibling Table* mapping shared logic across call sites; (2) **Expert A (Code Synthesizer)**: Synthesizes structural code edits using line anchors, error logs, and constraints $\mathcal{E}_{\text{dom}}$.

Prober outputs serve as non-binding hints. Candidate patch generation $\hat{p}$ requires dual-expert validation:

$$\hat{p} = \text{Synth}_A((x, g, e), \mathcal{E}_{\text{dom}}), \quad \text{Accept}(\hat{p}) = \bigwedge_{k \in \{A, B\}} \text{Verify}_k(\hat{p}, \mathcal{E}_{\text{dom}}), \quad (5)$$

Algorithm 1: Scientific Probing Procedure

Input : Artifacts $\mathcal{A}$, Baseline tests T_0 , Knowledge $\mathcal{K}$
Output: Contracts $\mathcal{C}$, Specs $\mathcal{P}_{\text{spec}}$, Probe tests T_P

```

1  $\mathcal{E} \leftarrow \text{COLLECTEVIDENCE}(\mathcal{A}, T_0, \mathcal{K});$ 
2  $\mathcal{C} \leftarrow$ 
    $\text{VALIDATECONTRACTS}(\text{DISCOVERCONTRACTS}(\mathcal{E}), \mathcal{E});$ 
3  $\mathcal{F} \leftarrow \text{INFERRFAILUREMODES}(\mathcal{C}, \mathcal{A}); \quad \mathcal{P}_{\text{gen}} \leftarrow \emptyset;$ 
4 foreach  $c \in \mathcal{C}$  do
5    $\mathcal{P}_{\text{gen}} \leftarrow \mathcal{P}_{\text{gen}} \cup \text{GENERATEALLPROBES}(c, \mathcal{K}, \mathcal{F});$ 
6  $\mathcal{P}_{\text{spec}} \leftarrow \text{DEDUPLICATE}(\text{VALIDATEPROBES}(\mathcal{P}_{\text{gen}}, \mathcal{C}));$ 
7  $T_{\text{exec}} \leftarrow$ 
    $\text{VALIDATEEXECUTABILITY}(\text{INSTANTIATETESTS}(\mathcal{P}_{\text{spec}}));$ 
8  $T_P \leftarrow \text{PRIORITIZEBYCOVERAGE}(T_{\text{exec}}, \mathcal{C});$ 
9 return  $(\mathcal{C}, \mathcal{P}_{\text{spec}}, T_P);$ 

```

where x denotes target files, g line anchors, and e the error log.

C. Reviewer: Domain-Aware Patch Evaluation

Reviewer evaluates candidate patches $\mathcal{P} = \{P_1, \dots, P_n\}$ across six dimensions $Q(P_i) = (F_i, N_i, S_i, R_i, I_i, M_i)$ covering functional correctness (F), numerical stability (N), scientific validity (S), robustness (R), intent preservation (I), and quality (M). The aggregate score is $\text{Score}(P_i) = \sum_k w_k \cdot X_{i,k}$.

By enforcing hard physical constraints $C(P_i) \in \{0, 1\}$, optimal patch selection is formulated as:

$$P^* = \arg \max_{P_i \in \mathcal{P}, C(P_i)=1} \text{Score}(P_i). \quad (6)$$

Few-shot evaluation leverages three-layer *Expert Shots* (Contract+Failure Pattern+Gold Pattern) to enforce semantic invariant alignment rather than surface code matching.

IV. SELF-EVOLVING

Existing LLM-based repair agents typically discard repair experience after each task, although such experience may contain reusable information about fault localization, scientific contracts, and repair strategies. Inspired by recent work on scientific experience consolidation [?], SPICE converts verified repair experience into reusable knowledge and applies it to subsequent tasks:

$$\mathcal{E}_t \rightarrow \mathcal{K}_t \rightarrow \mathcal{A}_{t+1} \rightarrow \mathcal{E}_{t+1}, \quad (7)$$

where $\mathcal{E}_t$, $\mathcal{K}_t$, and $\mathcal{A}_{t+1}$ denote repair experience, extracted knowledge, and the evolved repair agent, respectively.

a) *Experience and knowledge abstraction.*: For a task x , SPICE records candidate patches, execution outcomes, scientific contracts, Reviewer assessments, and failure evidence. Verified experience is represented as

$$\mathcal{E}_t = \{(x, p, e(p), C, F) \mid p \text{ is sufficiently validated}\}, \quad (8)$$

where C denotes scientific contracts and F denotes observed failures. A patch is not considered reusable solely because it passes tests, as it may overfit or violate scientific requirements.

An evolution module Φ abstracts verified experiences into reusable repair skills:

$$\mathcal{K}_t = \Phi(\mathcal{E}_t) = \{k_1, k_2, \dots, k_r\}, \quad (9)$$

where each skill

$$k_j = (\Gamma_j, \Omega_j, \Pi_j, \Lambda_j, \Delta_j) \quad (10)$$

captures its applicability, underlying mechanism, repair procedure, known failure patterns, and supporting evidence. This representation separates transferable repair mechanisms from repository-specific patches.

b) *Validation-gated evolution.*: To prevent incorrect knowledge accumulation, candidate skills are evaluated on tasks disjoint from their source experience. Their transfer utility is

$$U(\tilde{k}; \mathcal{D}_{\text{val}}) = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{x \in \mathcal{D}_{\text{val}}} [S(A(x; \tilde{k})) - S(A(x))], \quad (11)$$

where $S(\cdot)$ denotes the scientific repair score. A skill is incorporated only when it improves transfer performance without introducing regressions:

$$k^* = \begin{cases} \tilde{k}, & U(\tilde{k}; \mathcal{D}_{\text{val}}) > 0 \wedge R(\tilde{k}) \geq \tau_R, \\ k_{\text{old}}, & \text{otherwise,} \end{cases} \quad (12)$$

where $R(\tilde{k})$ measures regression safety.

c) *Contract-aware evolution.*: Scientific contracts provide an additional criterion for knowledge consolidation. Let

$$C(x) = \{c_1, c_2, \dots, c_n\} \quad (13)$$

be the contracts discovered by the Prober. A repair experience is considered valid only when the patch executes successfully and satisfies the required contracts:

$$\text{Valid}(p, x) = \text{Exec}(p, x) \wedge \bigwedge_{i=1}^n [v_i(p, x) \geq \tau_i], \quad (14)$$

where $v_i(p, x)$ denotes the satisfaction of contract c_i . This prevents the system from consolidating patches that merely pass conventional tests while violating scientific semantics.

d) *Agent update and reuse.*: Validated knowledge is incorporated into the subsequent repair process through

$$\mathcal{A}_{t+1} = \text{Update}(\mathcal{A}_t, \mathcal{K}_t^*). \quad (15)$$

SPICE does not require updating the underlying LLM parameters; instead, the agent evolves through validated procedural knowledge and scientific contracts. For a new task, relevant skills are retrieved using the task context, contracts, and observed failures:

$$\mathcal{K}(x) = \text{Retrieve}(x, C(x), F(x), \mathcal{K}^*). \quad (16)$$

The complete process is summarized as

$$\boxed{\mathcal{A}_t \xrightarrow{\text{repair}} \mathcal{E}_t \xrightarrow{\text{abstract}} \mathcal{K}_t \xrightarrow{\text{validate}} \mathcal{K}_t^* \xrightarrow{\text{update}} \mathcal{A}_{t+1}} \quad (17)$$

Thus, SPICE evolves from verified scientific repair experience rather than simply replaying or memorizing previous trajectories.

V. EXPERIMENTS

A. Experimental Setup

a) *Baselines & Models.*: All methods leverage the official DeepSeek API with external queries (`curl`) disabled. We evaluate SPICE against three representative baselines: single-agent repair via **DeepSeek Pro**, multi-agent framework **MAGIS** [3], and localization-driven **Agentless** [4]. For fair comparison, baselines enforce maximum reasoning effort on DeepSeek Pro. In contrast, SPICE employs an asymmetric allocation: a lightweight **DeepSeek Flash** (lower reasoning) for Probing, and **DeepSeek Pro** (max reasoning) for Patching, decoupling evidence extraction from patch synthesis.

b) *Dataset & Benchmark.*: We evaluate on an empirical subset of 30 tasks from SWE-BENCH SCIENCE (119 tasks), covering computational chemistry, materials simulation, and physics/engineering software (Table I). Each task pairs a public reproduction script with a held-out private test suite, isolating *false-green-light* failures where repairs pass public scripts but fail private tests. The subset retains all 30 tasks with recorded verifier runs (excluding two zero-pass tasks: 075, 094); 29 tasks exhibit false-green-light characteristics.

B. Correctness

Figure 2 reports the Pass@1 private test success rate (not the pass rate), where a candidate repair is considered correct only if it passes all held-out private test suites. Our proposed framework achieves a 57.1% Pass@1 rate, substantially outperforming DeepSeek Pro (35.2%), MAGIS [3] (38.2%), and Agentless [4] (43.8%). This represents a 13.3 percentage-point absolute improvement over the strongest baseline (Agentless). Notably, multi-agent architectures do not consistently outperform simpler single-agent baselines. Furthermore, fine-grained task analysis indicates that our agent encountered additional failures on private test cases for tasks 070 and 046 when compared to the vanilla DeepSeek Pro model.

C. Correctness

Figure 2 reports the average private test pass rate across evaluated tasks, where performance is measured by the mean proportion of passed private test cases per candidate repair. Our proposed framework achieves an average pass rate of 57.1%, substantially outperforming DeepSeek Pro (35.2%), MAGIS [3] (38.2%), and Agentless [4] (43.8%). This represents a 13.3 percentage-point absolute gain over the best baseline (Agentless). Notably, complex multi-agent architectures do not consistently surpass single-agent baselines. Fine-grained analysis further reveals minor trade-offs: SPICE exhibits slight regressions on private test suites for tasks 070 and 046 compared to vanilla DeepSeek Pro.

VI. RELATED WORKS

The emergence of large language models (LLMs) has transformed automated program repair (APR) from template-based transformation toward learned code generation and iterative refinement [5]–[11]. Recent LLM4APR systems increasingly

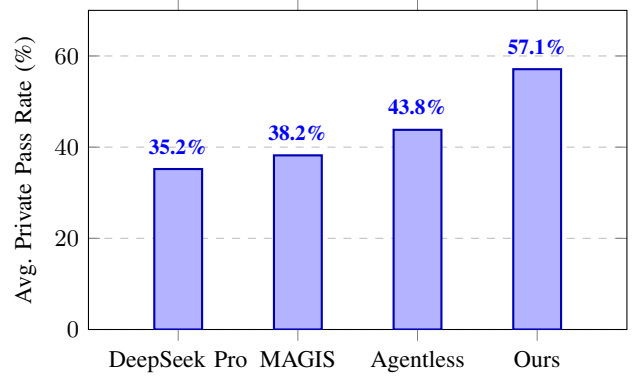


Fig. 2. Average private test case pass rates across evaluated benchmarks. SPICE outperforms the strongest baseline (Agentless) by 13.3 percentage points.

exploit retrieval, interactive feedback, and iterative patch refinement to improve repair effectiveness [12]–[15]. In parallel, recent studies on LLM self-improvement demonstrate that models can iteratively generate, evaluate, and refine their own outputs, although the reliability of self-generated feedback remains an important challenge [16]–[18]. These advances suggest a broader shift from one-shot generation toward iterative and feedback-driven problem solving. However, existing LLM4APR approaches largely employ a monolithic reasoning process, leaving the potential of specialized agents that independently generate evidence, patches, and evaluations underexplored.

VII. CONCLUSION AND FUTURE WORK

A. Conclusion and Future Work

In this work, we propose SPICE, a novel multi-agent framework grounded in contract-based verification tailored for scientific software repair. By decoupling lightweight evidence probing from high-reasoning patch synthesis, SPICE effectively mitigates false-green-light failures inherent in scientific codebases. Extensive empirical evaluation on SWE-BENCH SCIENCE demonstrates that our approach achieves an average private test pass rate of 57.1%, outperforming the state-of-the-art Agentless baseline by 13.3 percentage points. Ultimately, our mission is to provide long-term, dependable software ecosystem maintenance for scientific researchers through persistent agent memory and continuous self-evolution.

Looking ahead, we highlight three critical directions for future work: First, we plan to scale our evaluation by mining broader, real-world scientific datasets from the open-source community. Second, we will systematically benchmark a wider spectrum of multi-agent patching frameworks to deepen our understanding of agent coordination dynamics. Finally, we aim to explore advanced self-evolution paradigms alongside long-term memory architectures, investigating their stability and cumulative efficacy over extended deployment horizons.

TABLE I
EVALUATED SWE-BENCH SCIENCE SUBSET (30 TASKS). [†] DENOTES NON-PYTHON CODE (FORTRAN/C/OCTAVE); [‡] DENOTES DOCUMENTED FALSE-GREEN-LIGHT CASES.

Domain (<i>N</i>)	Task IDs
Chemistry (9)	001, 002, 064, 070, 079, 085 ^{†‡} , 089 [‡] , 090 [‡] , 119
Materials (7)	003, 021, 023, 082, 086 [‡] , 095, 109
Physics/Eng. (14)	005 [†] , 041, 046, 050, 063 [†] , 071, 072, 087, 088, 113–117

REFERENCES

- [1] M. Tian, L. Gao, S. D. Zhang, X. Chen, C. Fan, X. Guo, R. Haas, P. Ji, K. Krongchon, Y. Li, S. Liu, D. Luo, Y. Ma, H. Tong, K. Trinh, C. Tian, Z. Wang, B. Wu, Y. Xiong, S. Yin, M. Zhu, K. Lieret, Y. Lu, G. Liu, Y. Du, T. Tao, O. Press, J. Callan, E. Huerta, and H. Peng, “Scicode: A research coding benchmark curated by scientists,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.13168>
- [2] Z. Xu, J. Lu, Y. Zheng, Y. Wang, and X. Qiu, “Swe-bench science: Can coding agents resolve engineering tasks in science?” 2026. [Online]. Available: <https://arxiv.org/abs/2608.19799>
- [3] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng, “Magis: Llm-based multi-agent framework for github issue resolution,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 51 963–51 993.
- [4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Demystifying llm-based software engineering agents,” *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3715754>
- [5] N. Jiang, T. Lutellier, and L. Tan, “Cure: Code-aware neural machine translation for automatic program repair,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 1161–1173.
- [6] D. Drain, C. Wu, A. Svyatkovskiy, and N. Sundaresan, “Generating bug-fixes using pretrained transformers,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, 2021.
- [7] Y. Hu, X. Shi, Q. Zhou, and L. Pike, “Fix bugs with transformer through a neural-symbolic edit grammar,” *arXiv preprint arXiv:2204.06643*, 2022.
- [8] Y. Wang *et al.*, “T5apr: Empowering automated program repair across languages through checkpoint ensemble,” *Journal of Systems and Software*, vol. 214, p. 112083, 2024.
- [9] W. Wang, Y. Wang, S. Joty, and S. C. H. Hoi, “Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 2023, pp. 146–158.
- [10] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, “Inferfix: End-to-end program repair with llms over retrieval-augmented prompts,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 2023.
- [11] A. Silva, S. Fang, and M. Monperrus, “Repairllama: Efficient representations and fine-tuned adapters for program repair,” *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2366–2380, 2025.
- [12] C. S. Xia and L. Zhang, “Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 2024, pp. 819–831.
- [13] X. Yin, C. Ni, S. Wang, Z. Li, L. Zeng, and X. Yang, “Thinkrepair: Self-directed automated program repair,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 2024, pp. 1274–1286.
- [14] J. Xiang, X. Xu, F. Kong, M. Wu, Z. Zhang, H. Zhang, and Y. Zhang, “How far can we go with practical function-level program repair?” *arXiv preprint arXiv:2404.12833*, 2024.
- [15] J. Xu, Y. Fu, S. H. Tan, and P. He, “Aligning the objective of llm-based program repair,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.
- [16] A. Huang, A. Block, D. J. Foster, D. Rohatgi, C. Zhang, M. Simchowitz, J. Ash, and A. Krishnamurthy, “Self-improvement in language models: The sharpening mechanism,” in *NeurIPS Workshop*, 2024.
- [17] D. Jiang, J. Zhang, O. Weller, N. Weir, B. Van Durme, and D. Khashabi, “Self-[in]correct: Llms struggle with discriminating self-generated responses,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.
- [18] J. Wang, Y. Zhou, X. Zhang, M. Bao, and P. Yan, “Self-evolutionary large language models through uncertainty-enhanced preference optimization,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.